FINE-TUNING LLMs

Priyadarshini Priya
Scientific Officer/E
Computer Division, BARC

Defining LLMs & Pretraining

- Large Language Models (LLMs) are transformer-based neural networks trained on trillions of tokens to understand and generate human language.

- Pretraining is the first and most resource-intensive phase of creating a Large Language Model.

- During pretraining, a model is shown a massive amount of unlabeled text (trillions of words from the internet, books, and code), example “the pile” by **EleutherAI.**
- By doing this billions of times, the model implicitly learns:
  - Grammar: How to structure sentences.
  - Facts: Information about the world (e.g., "The capital of France is Paris").
  - Reasoning: Patterns of logic and common sense.
  - Coding: Syntax of various programming languages.
  - Language (the one in which it is trained)

Limitations of Pretrained model

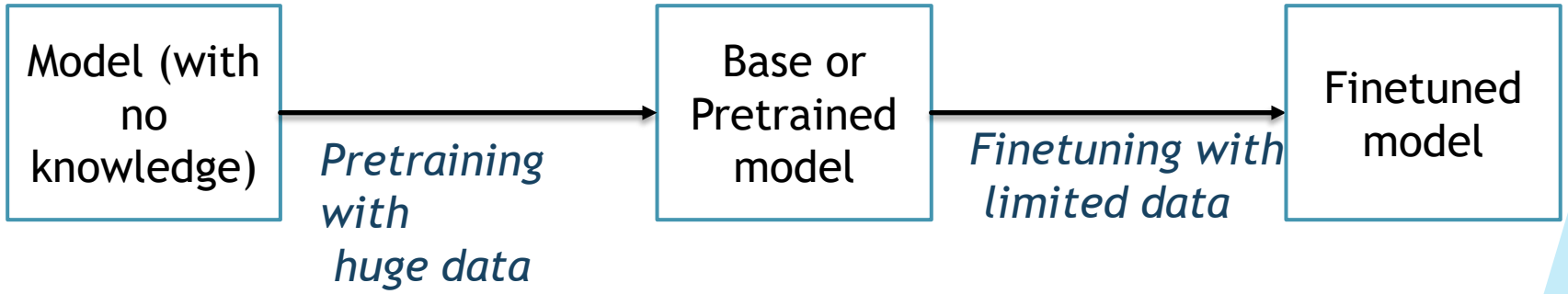
- The "*Knowledge Cutoff*“: A model pretrained in 2023 has no inherent knowledge of events, technologies, or research that occurred in 2024.
- Lack of Domain Expertise
- Tone, Style, and Format Alignment
- Reliability and Hallucination Control

Prompt Engineering V/S Finetuning:

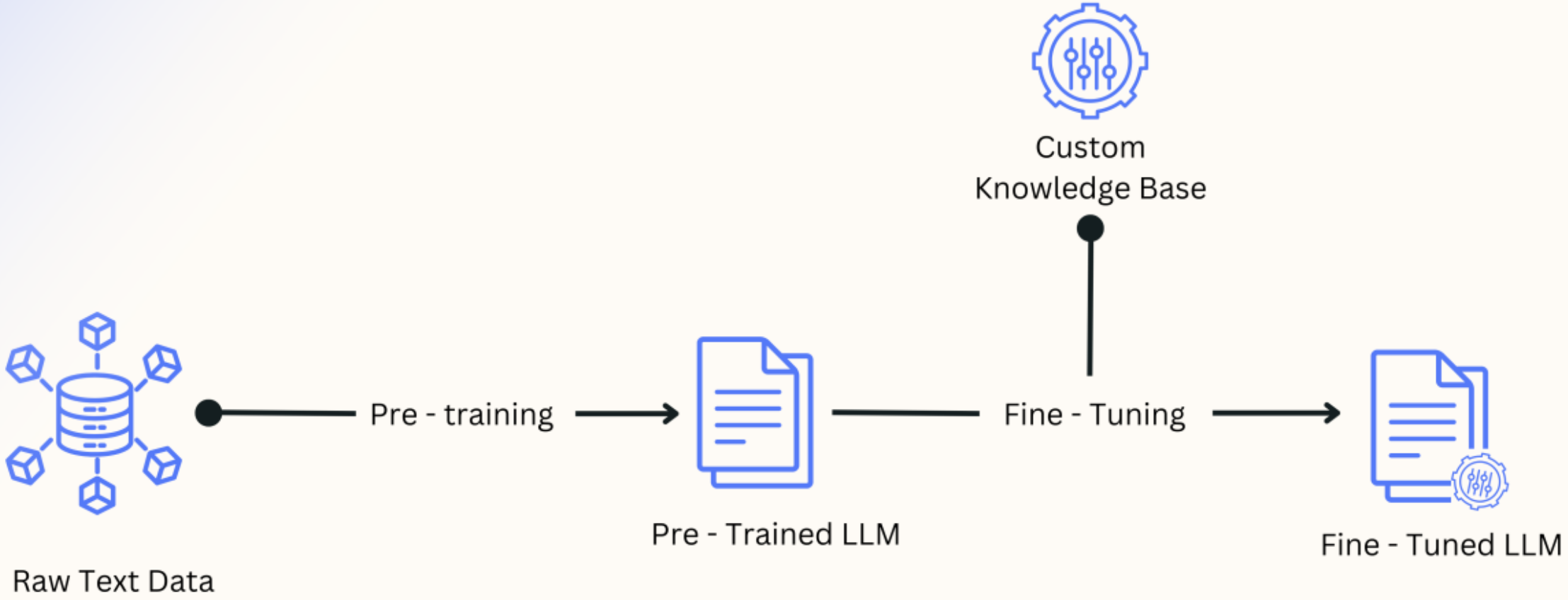
	Prompt engineering	Finetuning
Mechanism	Instruction in Input (Frozen weights)	Weight Modification
Data needs	1-5 examples in prompt	100+ curated labels
Knowledge	Context-window based	Internalized in model
Best for	Prototypes & General Tasks	Domain Mastery & Reliability

Finetuning LLMs

- Fine-tuning is the process of taking a pre-trained model and further training it on a smaller, specific dataset to adapt it for a particular task or domain.
- Python libraries for finetuning:Hugging Face ‘transformers’ , peft (Parameter-Efficient Fine-Tuning) ,etc

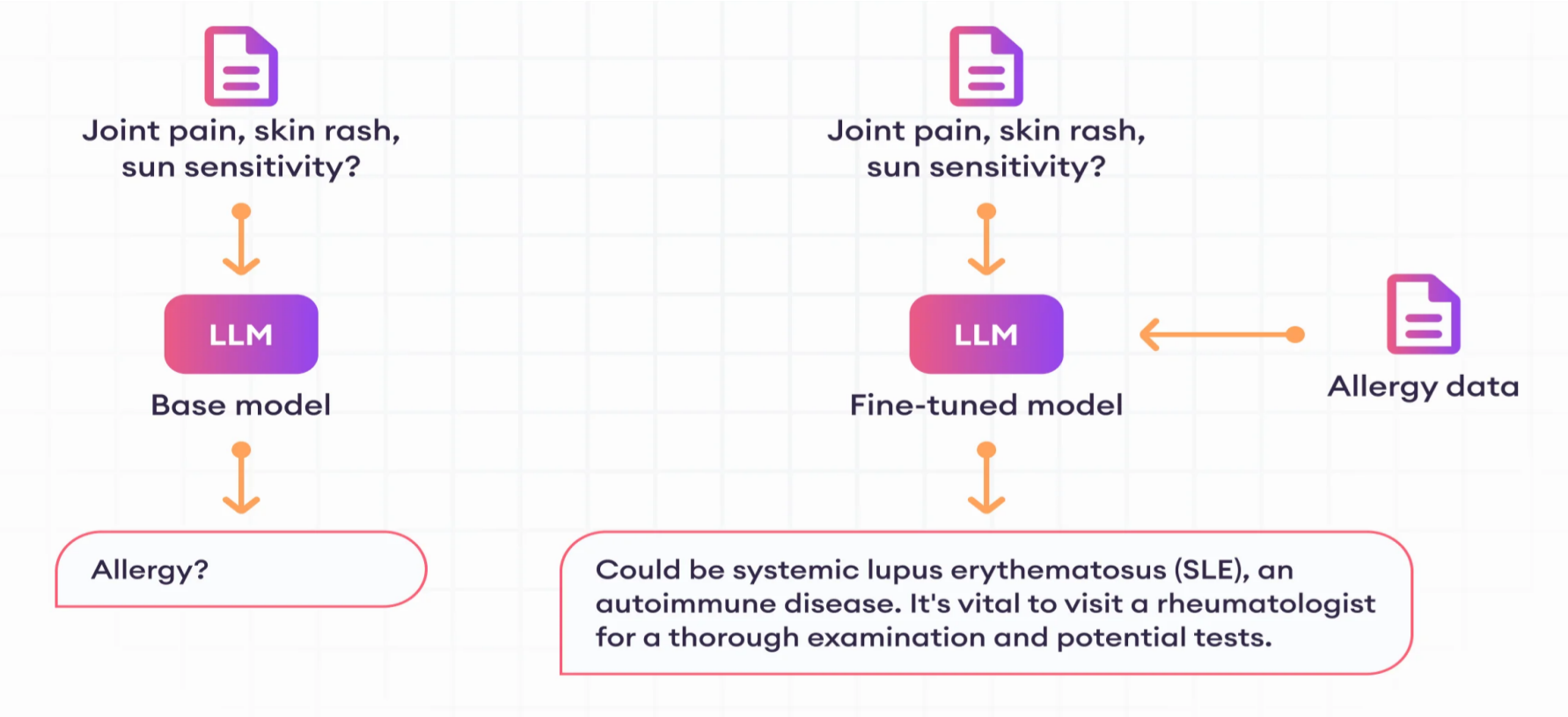


FINETUNING PROCESS

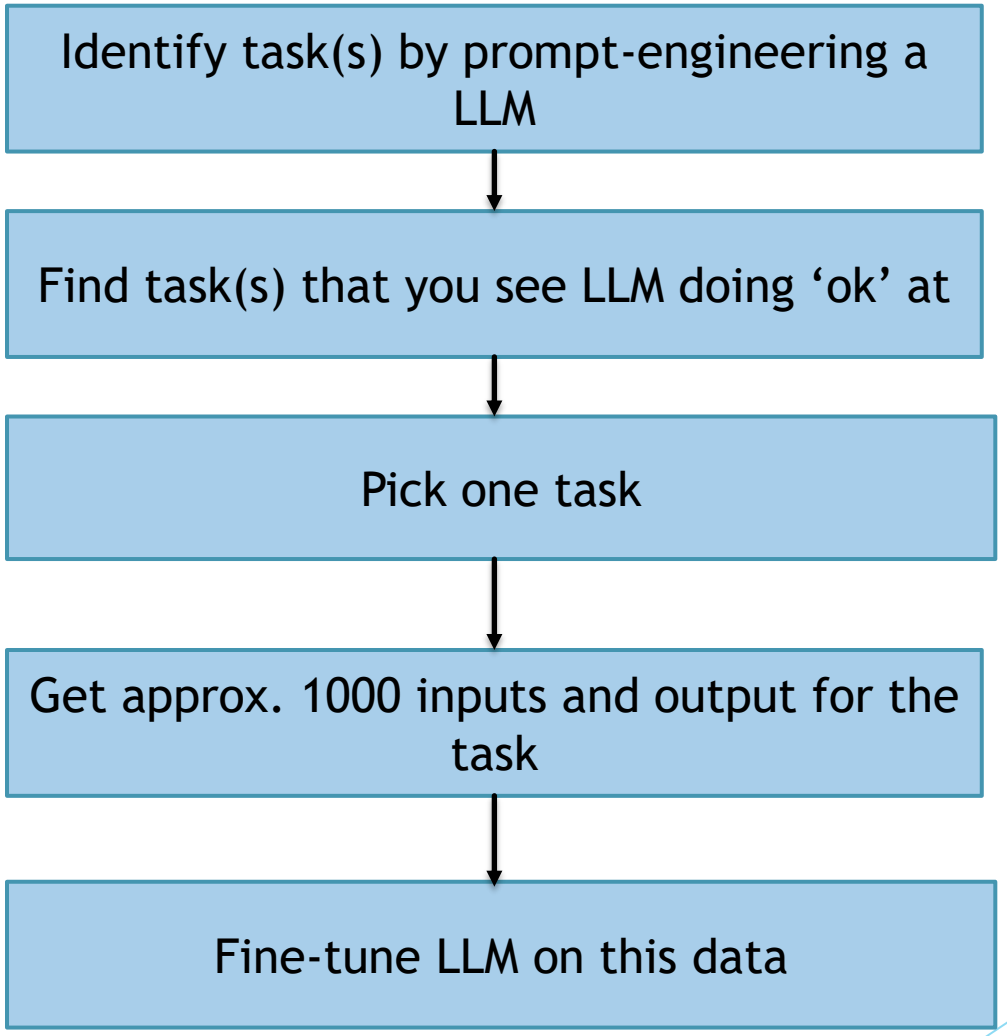


What does Fine-tuning do:

- Lets you add more data to model than what fits in prompt.
- Gets model to learn data and thus updates internal model weights.
- Steers model to consistent output about a particular domain.
- Reduces hallucinations.



First time fine-tuning:



Data Preparation (Prompt templates):

- Every model has a unique ‘Special Token’ set. During fine-tuning, you MUST match the template the model was originally aligned with. Example:
  - Mistral: Uses [INST] and [/INST].
  - Llama 3: Uses <|start_header_id|> tags.
  - Alpaca: Uses text headers like ### Instruction:.
- *Using the wrong template leads to "garbage output" because the model doesn't recognize the instructions.*

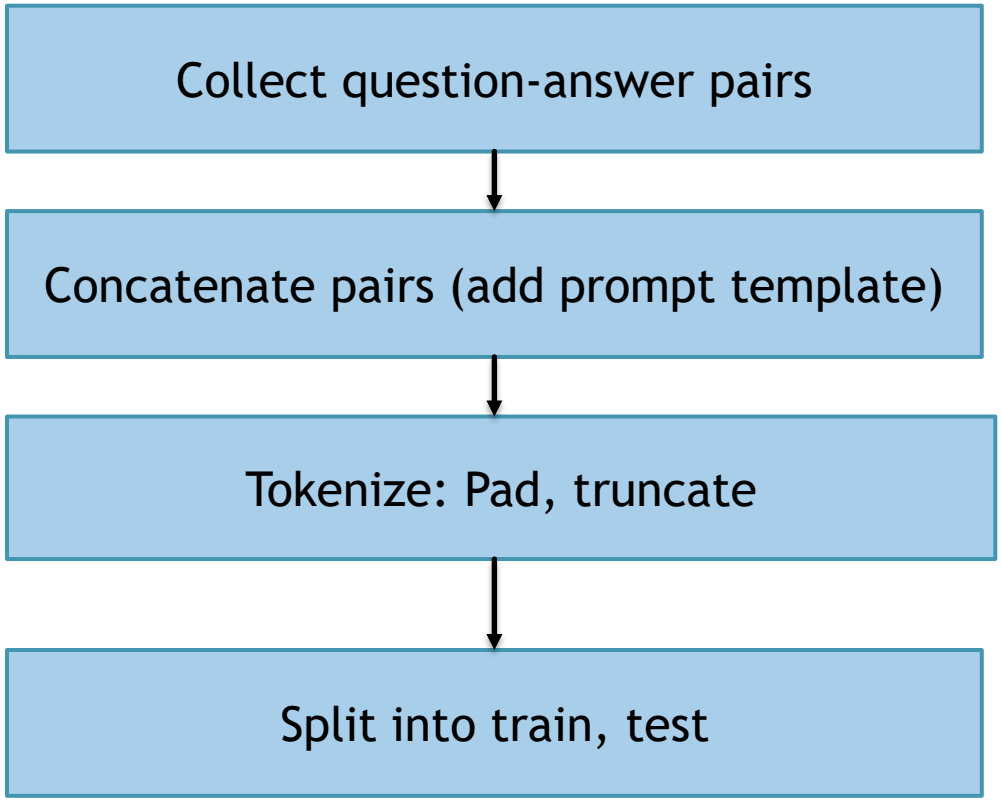
LLAMA 3 TEMPLATE EXAMPLE:

<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are an expert...<|eot_id|>
<|start_header_id|>user<|end_header_id|>
What is ANUPAM? <|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
ANUPAM is a supercomputer...<|eot_id|>

Kind of data:

Better	Worse
Higher quality (no garbage)	Lower quality
Diversity	Homogeneity
Real	Generated
More	Less

Steps to prepare data:



Training the LLM:

Add training data to model → Calculate loss → Back propagation through model → Update weights

Training Hyperparameters:

Fine-tuning success depends on tuning the "knobs" that control how the model learns

Learning Rate:
Controls the step size for weight updates. Typically 2e-4 for LoRA. Too high causes instability; too low slows learning.

Batch Size:
*Nu*mber of samples processed before updating weights. Larger batches provide stable learning but require more VRAM.

Optimizer:
The math engine (e.g., *AdamW 8-bit)* that calculates weight changes. 8-bit versions save significant memory.

Epochs / Steps:
Determines how many times the model sees the entire data. Small datasets usually need 3-5 epochs to memorize facts.

LoRA Rank (r):
Determines the "Rank" of the update matrices. Higher r (16, 32) captures more detail but increases memory usage.

- Some other training hyperparameters:

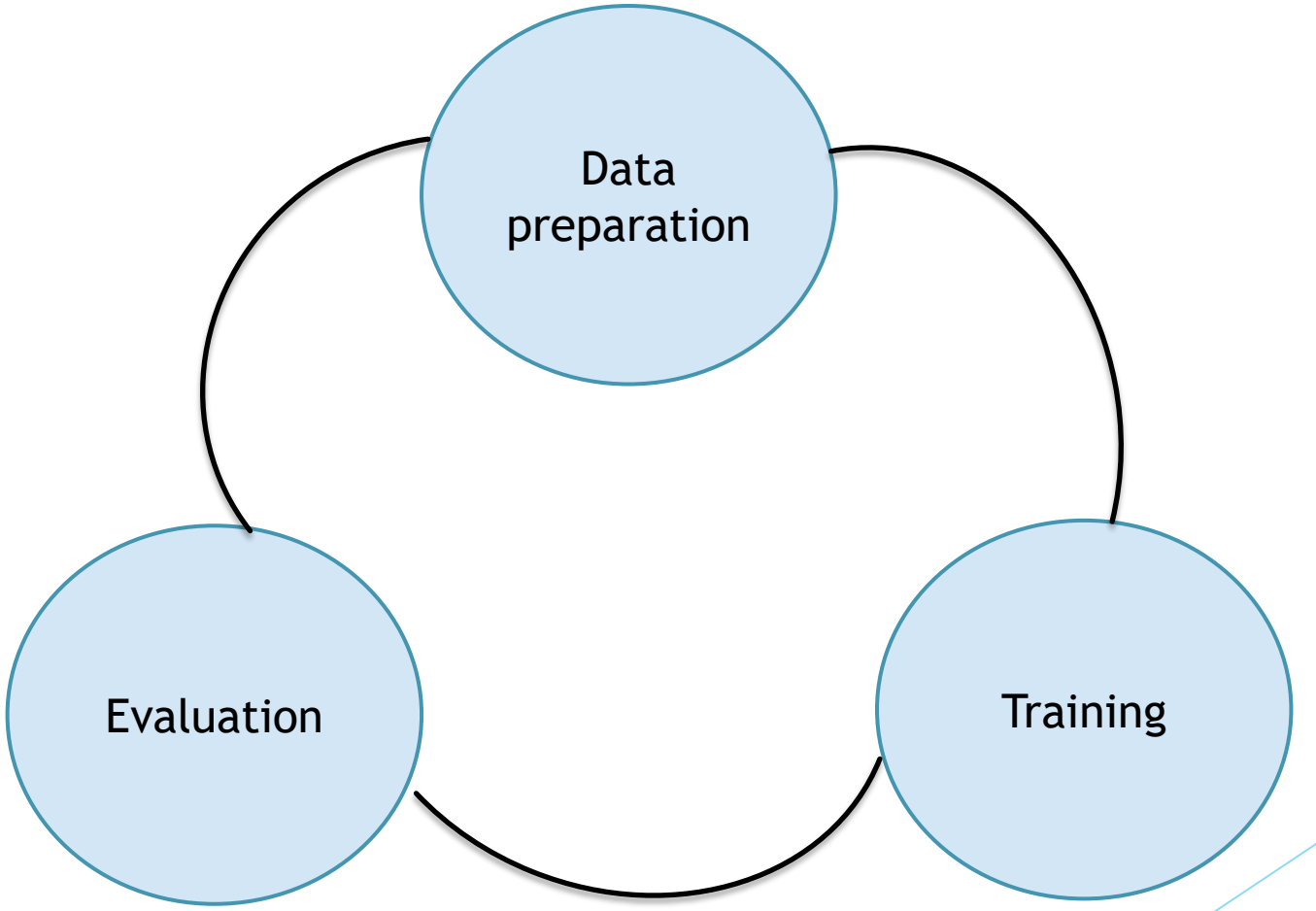
- **lora_alpha(α):** Alpha is the scaling factor for your LoRA adapters. If you set α high, the model will prioritize the new fine-tuning data very aggressively and if set low the model will lean more on its original pre-trained knowledge.
- **gradient_accumulation_steps** : It allows you to simulate a large batch size by splitting it into several smaller mini-batches and summing their gradients before updating the model weights. Ex: if batch size is 2 and gradient accumulation step is 8,model process 8 mini batches of 2 before updating weights.
- **lora_dropout**: the model randomly zeros out a percentage of the values in the adapter layers during each training step.

LLM benchmarks (Evaluation methods):

- In the context of Large Language Models (LLMs), ARC and HellaSwag are two of the most foundational benchmarks used to measure "reasoning.“

- While they both test how a model "thinks," they target very different cognitive abilities: one focuses on academic knowledge, while the other focuses on daily logic.

LLM fine-tuning:



LLM fine-tuning:

- Has three important concepts:
  - Numerical precision
  - Quantization
  - Modes of quantization

Numerical precision in LLMs:

- ‘Precision’ refers to the format used to represent the numbers (weights and activations) within a neural network.
- Choosing the right data type is a balance between memory usage, computational speed, and numerical stability.

Feature	FP32(full)	FP16(half)	BF16	INT8/NF4
Bits	32 bits	16 bits	16 bits	8/4 bits
Memory per param	4 Bytes	2 Bytes	2 Bytes	1 / 0.5 Bytes
Exponent (Range)	8 bits (High)	5 bits (Low)	8 bits (High)	NA
Mantissa (Precision)	23 bits (High)	10 bits (Mid)	7 bits (Low)	NA
Max Value	~3.4 *10^38	65,504	~3.4 *10^38	

Quantization:

- It is conversion from higher precision to lower precision.

- **Symmetric Quantization:**
  - The range is centered around zero. The floating-point range [xmin,xmax] is mapped to the integer range [−128,127] (for int8).

$$scale(S) = (xmax-xmin)/(qmax-qmin)$$

  - Mathematical Relation: It uses a single Scale Factor (S) which has relation as :

$$\mathbf{Q=round(x/S).}$$

Where Q is quantized value and x is original value.

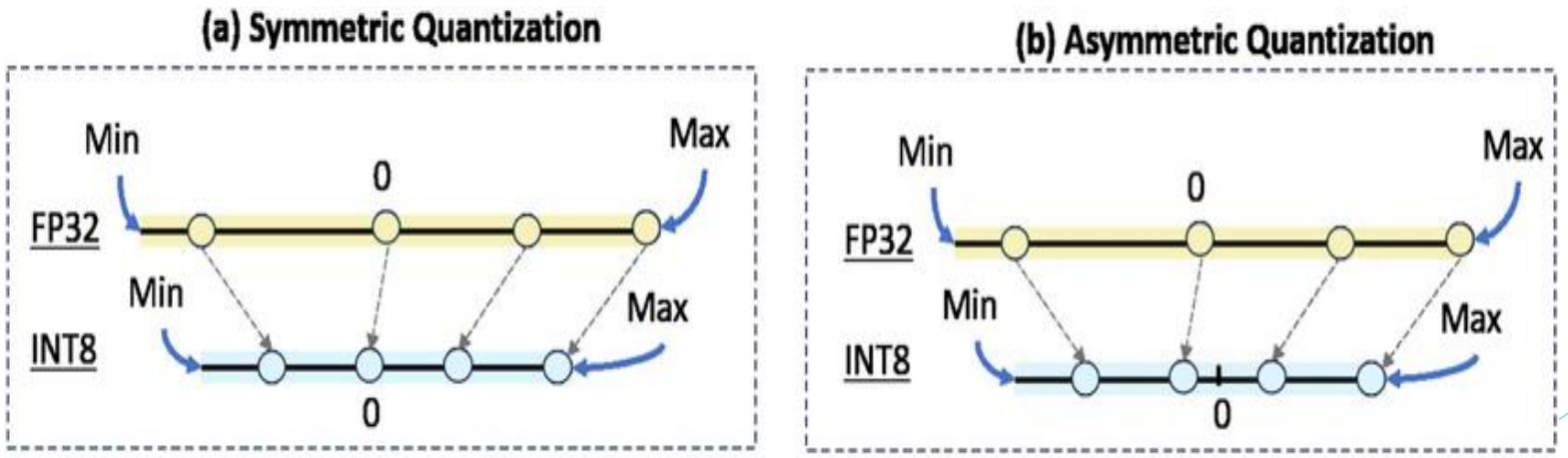
- **Asymmetric Quantization**
  - The range is mapped to the exact min/max of the data. The floating-point range [xmin,xmax] is mapped to the full integer range [0,255] (for uint8).

$$S=(xmin-xmax)/(qmax-qmin)$$

$$Z=int(round(qmin-xmin/S)$$

  - Mathematical Relation: It uses both a Scale Factor (S) and a Zero Point (Z) with formula as

$$Q=round(x/S+Z).$$



Modes of Quantization:

- Post Training Quantization (PTQ)
- Quantization Aware Training (QAT)

Post Training Quantization (PTQ):

- Use a trained model, then apply quantization to it to reduce memory footprint.
- Leads to loss of accuracy of trained model due to quantization process.

Quantization Aware Training (QAT):

▶ LoRA(Low-rank adaptation):

Low-rank adaptation (LoRA) is a parameter-efficient fine-tuning (PEFT) technique that adds a subset of parameters to the frozen base foundation model and updates the subset during the tuning experiment, without modifying the parameters of the base model.

❖ LoRA Matrix Decomposition:

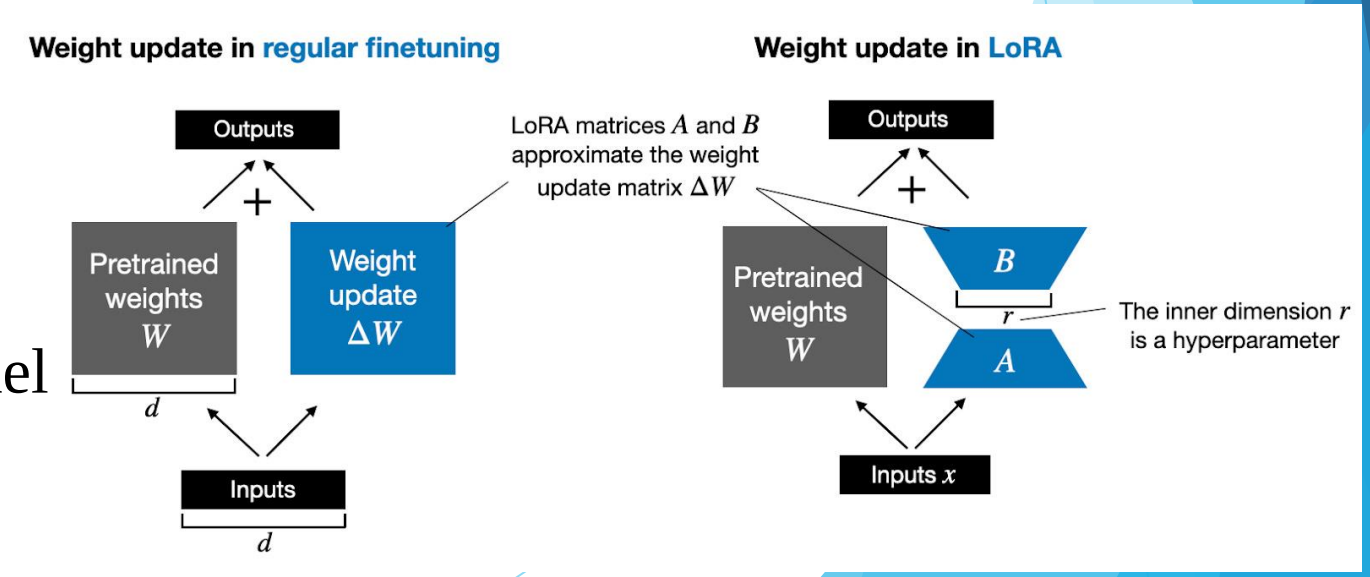
LoRA freezes the pre-trained weight matrix W and injects trainable low-rank matrices (adapters).

$$W' = W_{frozen} + \Delta W = W_{frozen} + B \cdot A$$

Here :

W_{frozen} is frozen weights of base model

A & B are adapter weights



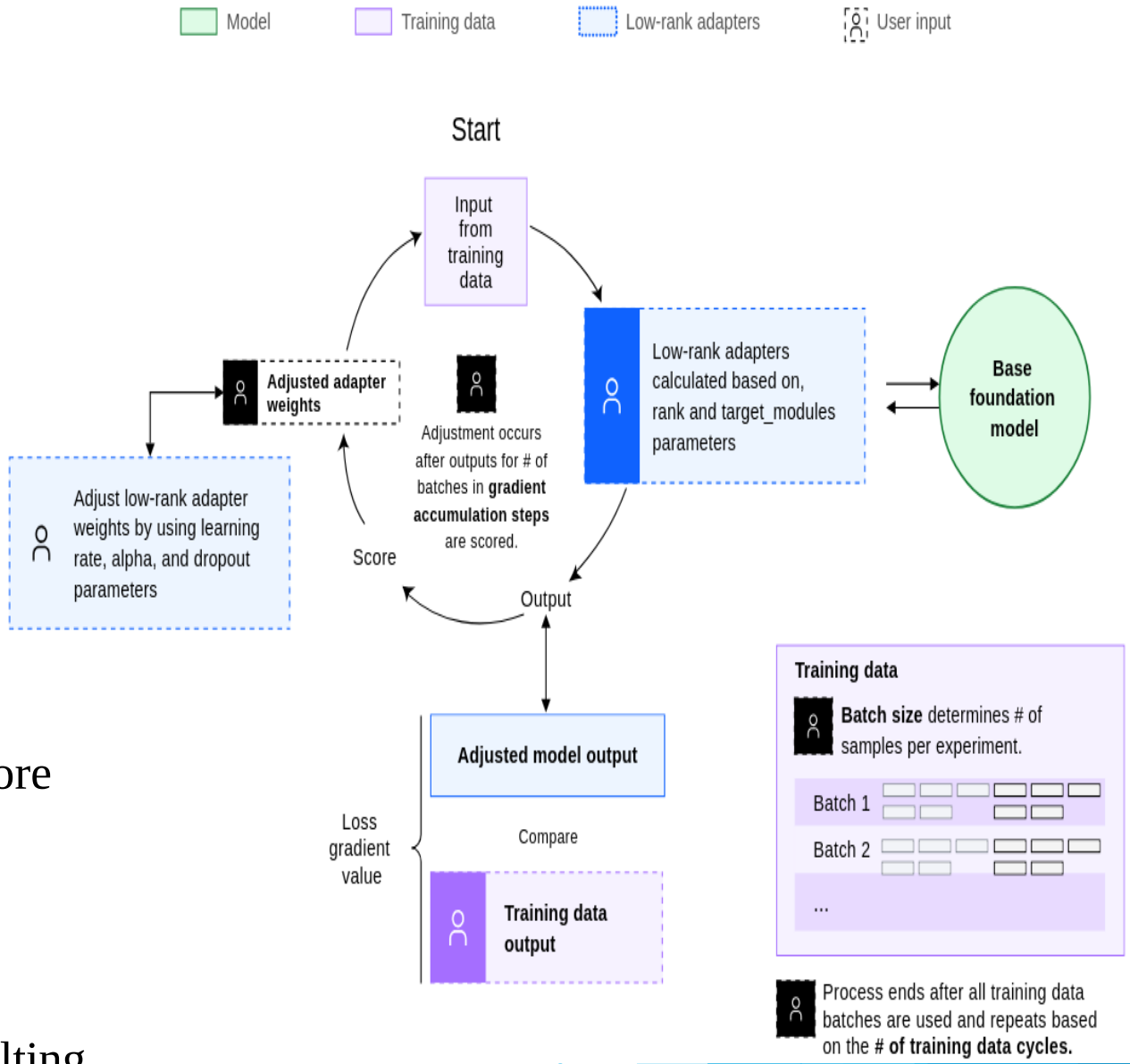
$$W' = W_{frozen} + \Delta W = W_{frozen} + B \cdot A$$

1. When training starts, $\Delta W = 0 = A*B$

If W matrix is (d*k) format, then:
*A(d*r):*Filled with random numbers (Gaussian noise).
B(r*k): Filled entirely with 0

2. During first forward pass for a layer, hidden state equation is : $h = W_{frozen}x + \left(\frac{\alpha}{r}\right)\Delta W x$

Since $\Delta W=0$ initially, model output is exactly same as before fine tuning during first forward pass.
3. The Loss function calculates difference between the predicted output and the actual output.
4. Backpropagation calculates gradient for A and B.
5. The optimizer uses the gradients to update A and B resulting in nonzero ΔW.
6. The process repeats until all of the batches and epochs covered



Pros of LoRA:

- Extreme memory efficiency: Because only small adapter matrices are trained and not billions of parameters, we don’t need to store gradients for entire model.

- Small portability footprint: When finished training, the output is just the adapter weights.

- Reduced Risk of Catastrophic Forgetting: In full fine-tuning, the model can "overwrite" its general reasoning capabilities while trying to learn your specific data.

Fine-tuning V/S LoRA:

Feature	Full Fine-Tuning	LoRA
Trainable Parameters	100%	~1% – 5%
Memory Requirement	Very High (Multiple A100s)	Low (Single Consumer GPU)
Storage Cost	Full copy of the model per task	Tiny adapter files
Training Speed	Slower	Faster
Risk of Catastrophic Forgetting	High (can "unlearn" base knowledge)	Low (base weights are frozen)

What is QLoRA:

- QLoRA = Quantized Low-Rank Adaptation.
- QLoRA (Quantized LoRA) works by loading the base language model in a highly compressed 4-bit quantized format, drastically reducing memory usage, while training small LoRA adapters in higher precision.
- An extension of LoRA that reduces memory usage significantly.

- ***Training process:***
- The pretrained model is loaded with quantized 4-bit weights.
- Only the LoRA adapters are updated during training.
- Libraries like BitsAndBytes (for quantization) and PEFT (for LoRA) are used together for implementation.

Key Features of QLoRA:

- **Further Memory Reduction**: QLoRA extends LoRA by quantizing the main model weights to 4 bits (using methods like NF4), while keeping the LoRA adapters in higher precision (e.g., 16-bit).
- **Ultra-Low Resource Requirements**: QLoRA can fine-tune very large models (billions of parameters) on consumer-grade GPUs or even CPUs by reducing VRAM needs to as little as 0.5GB per 1GB model.
- **Performance**: QLoRA has been shown to maintain comparable accuracy to standard LoRA and full fine-tuning, even on very large models.
- **Double Quantization**: QLoRA may use double quantization techniques to further compress storage, especially for scale/offset constants.
- **Trade-offs:** While QLoRA is slightly slower than LoRA due to quantization/dequantization steps, the memory savings are substantial, and the method is highly scalable.